

Reviewer Pack — Minimal Rank of Noncommutative Representations and Circuit M...

Entry e5ec85752d00 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 16:30:34 UTC

Minimal Rank of Noncommutative Representations and Circuit Monotone Width

Entry ID: e5ec85752d00 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 16:30:34 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Geometry **Field B** (complexity object): Boolean Circuits

Statement:

For any given Boolean circuit C with n inputs, the minimal rank r of a noncommutative representation associated with C is linearly correlated with its monotone width $w_m(C)$, such that $r = \Theta(w_m(C))$. Furthermore, for all circuits with monotone width greater than some threshold T , their associated noncommutative representation will have a rank at least as large as the circuit's monotone width.

Rationale (proposer's reasoning):

Noncommutative geometry has recently been applied to quantum information theory and its geometric interpretations may also reveal insights into the structure of Boolean circuits. The minimal rank of a noncommutative representation could serve as an invariant that captures essential complexity properties of circuits, such as their monotone width, providing a bridge between these two fields.

Taxonomy category: Noncommutative_Geometry_x_Boolean_Circuits (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): fa3fceabf81aac84

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

A conjecture is supported if, for all circuits with n inputs ($n \leq 40$), the minimal rank r of a noncommutative representation is within $\Theta(w_m(C))$, where $w_m(C)$ is the monotone width of the circuit, and at least 30 independent random seeds yield a `support_fraction` ≥ 0.8 AND `metric_mean` ≤ 3 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "noncommutative geometry" AND "Boolean circuit monotone width" - "minimal rank noncommutative representations" AND "circuit monotone width" - "correlation minimal rank noncommutative representations" AND "monotone width Boolean circuits"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.5s

5.1 Generated Python source

```
import random
from fractions import Fraction
from itertools import product

def generate_random_circuit(n):
    if n <= 1:
        return []

    gates = ['AND', 'OR', 'NOT']
    circuit = []
    for _ in range(2 * n - 2):
        gate = random.choice(gates)
        inputs = [random.randint(0, n-1) for _ in range(2)]
        circuit.append((gate, inputs))

    return circuit

def compute_monotone_width(circuit):
    if not circuit:
        return 0

    n = len(circuit) + 1
    width = [[0] * n for _ in range(n)]

    for i, (gate, inputs) in enumerate(circuit):
        if gate == 'NOT':
            width[inputs[0]][i+1] += 1
        else:
            width[inputs[0]][i+1] += 1
            width[inputs[1]][i+1] += 1
```

```

    for j in range(n-2, -1, -1):
        for i in range(j+1):
            width[i][j] = max(width[i][j], width[i][j+1])

    return width[0][0]

def compute_noncommutative_rank(circuit):
    n = len(circuit) + 1
    rank = [[0] * n for _ in range(n)]

    for i, (gate, inputs) in enumerate(circuit):
        if gate == 'NOT':
            rank[inputs[0]][i+1] += 1
        else:
            rank[inputs[0]][i+1] += 1
            rank[inputs[1]][i+1] += 1

    for j in range(n-2, -1, -1):
        for i in range(j+1):
            rank[i][j] = max(rank[i][j], rank[i][j+1])

    return sum(max(row) for row in rank)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        for _ in range(5):
            circuit = generate_random_circuit(n)
            w_m = compute_monotone_width(circuit)
            r = compute_noncommutative_rank(circuit)
            results.append((n, w_m, r))

    total_r = sum(r for _, _, r in results)
    total_w_m = sum(w_m for _, w_m, _ in results)
    instances_tested = len(results)
    n_max = max(n for n, _, _ in results)

    if instances_tested < 30:
        return {
            "metric_name": "rank",
            "metric_value": total_r / instances_tested,
            "instances_tested": instances_tested,
            "n_max": n_max,
            "conjecture_holds": False,
            "counterexample": "insufficient_instances"
        }

    mean_r = total_r / instances_tested
    mean_w_m = total_w_m / instances_tested

    if abs(mean_r - mean_w_m) > 3:
        return {

```

```

        "metric_name": "rank",
        "metric_value": mean_r,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": f"mean_r={mean_r}, mean_w_m={mean_w_m}"
    }

support_fraction = sum(1 for _, _, r in results if abs(r - mean_w_m) <= 3) / instances_tested

return {
    "metric_name": "rank",
    "metric_value": mean_r,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": support_fraction >= 0.8,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19,

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    total_r = sum(r["metric_value"] for r in results if "conjecture_holds" in r and r["conjecture_holds"])
    total_instances = sum(r["instances_tested"] for r in results if "conjecture_holds" in r and r["conjecture_holds"])

    support_fraction = total_r / total_instances if total_instances > 0 else 0

    if all("conjecture_holds" in r and r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={total_r/len(results)} std=0.0 support_fraction={support_fraction}")
    elif any("counterexample" in r and r["counterexample"] for r in results):
        counterexamples = [r["counterexample"] for r in results if "counterexample" in r]
        first_failing_seed = next(r["seed"] for r in results if "conjecture_holds" not in r or not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{' '.join(counterexamples)}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

ounterexample': 'mean_r=20.533333333333335, mean_w_m=1.0'}
TRIAL: {'metric_name': 'rank', 'metric_value': 20.433333333333334, 'instances_tested': 30, 'n_max': 40, 'conjecture_holds': False, 'counterexample': ''}
TRIAL: {'metric_name': 'rank', 'metric_value': 20.766666666666666, 'instances_tested': 30, 'n_max': 40, 'conjecture_holds': False, 'counterexample': ''}
TRIAL: {'metric_name': 'rank', 'metric_value': 20.233333333333334, 'instances_tested': 30, 'n_max': 40, 'conjecture_holds': False, 'counterexample': ''}
TRIAL: {'metric_name': 'rank', 'metric_value': 20.166666666666668, 'instances_tested': 30, 'n_max': 40, 'conjecture_holds': False, 'counterexample': ''}
TRIAL: {'metric_name': 'rank', 'metric_value': 20.366666666666667, 'instances_tested': 30, 'n_max': 40, 'conjecture_holds': False, 'counterexample': ''}
TRIAL: {'metric_name': 'rank', 'metric_value': 20.7, 'instances_tested': 30, 'n_max': 40, 'conjecture_holds': False, 'counterexample': ''}

```

```

TRIAL: {'metric_name': 'rank', 'metric_value': 20.533333333333335, 'instances_tested': 30, 'n_max': 4}
TRIAL: {'metric_name': 'rank', 'metric_value': 20.166666666666668, 'instances_tested': 30, 'n_max': 4}
TRIAL: {'metric_name': 'rank', 'metric_value': 20.933333333333334, 'instances_tested': 30, 'n_max': 4}

```

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4856de19.py", line 137, in <

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The conjecture was falsified as the test results show that for all tested instances, the minimal rank of noncommutative representations does not hold | next: Investigate further to understand why the conjecture fails for these instances and explore alternative methods or theories that may explain the observed behavior.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17526
2	propose	ollama_remote	glm4:latest	0	0	15289
3	propose	ollama_remote	glm4:latest	0	0	15342
4	preregistration	ollama_remote	glm4:latest	0	0	15115
5	novelty	ollama_remote	glm4:latest	0	0	17658
6	novelty	ollama_remote	glm4:latest	0	0	18882
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23588
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11490
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10180
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22917
11	judge	ollama_remote	glm4:latest	0	0	15895

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 183881 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/e5ec85752d00.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/e5ec85752d00.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/e5ec85752d00.tar.gz` (if generated)